

# 【专栏二：深度学习09】 - 【一张图讲清楚：过拟合是什么？模型为什么会学“过头”】



快乐飞猪ing



· 码龄7年 · 粉丝可见 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-30 00:38:12 · 773 阅读



AI 写同款 · 数据监测



A/推动工业智能化转型~系列文章09：深度学习基础：神经网络的工业适应性分析

本文系统分析了深度学习在工业领域的适用性与局限性。首先构建神经网络的核心知识框架，包括权重...

[阅读详情 >](#)

## 前言

一张图讲清楚：什么是过拟合？模型为什么会学“过头”？

很多人刚接触模型训练时，都会遇到一个很困惑的现象：

- 训练集表现越来越好 ✓
- 测试集表现却越来越差 !

甚至会出现：**训练准确率 99%，测试准确率只有 70%**，这并不是模型变差了，而是：**模型学“过头”了 (Overfitting)**

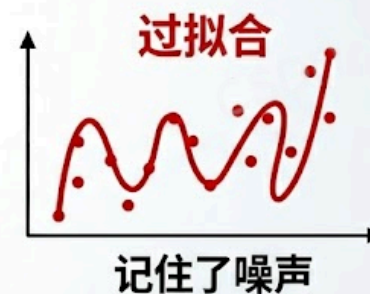
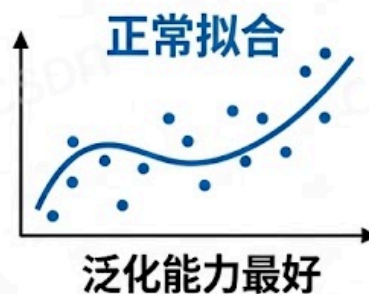
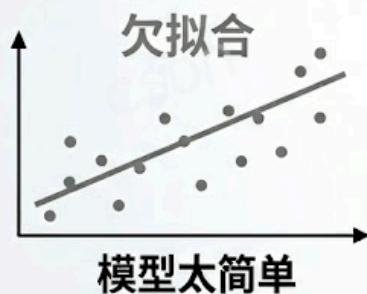
内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

# 一张图讲清楚：什么是过拟合（Overfitting）？



过拟合 = 学会了数据细节，但丢失了泛化能力

这张图其实讲了两件非常重要的事情：

1 上半部分：Loss 曲线（最关键）

- 蓝线（训练集 Loss）：一直下降
- 红线（验证集 Loss）：先下降 → 后上升

👉 关键转折点：验证集 Loss 开始上升的那一刻，就是过拟合开始的地方

2 下半部分：三种拟合状态

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

- 欠拟合：模型太简单，连规律都没学会
- 正常拟合：学到规律，泛化能力最好
- 过拟合：不仅学规律，还记住了噪声

👉 一句话先记住：**过拟合 = 学会了数据细节，但丢失了泛化能力**

## 一、从“**训练过程**”理解：模型到底在做什么？

很多人理解过拟合卡住的地方在于：

- ❌ 只看结果
- ✅ 没看训练过程

模型在训练时，其实只在做一件事：

$$\min_{\theta} \mathcal{L}_{train}(\theta)$$

也就是说：**模型的目标只有一个：让训练集 Loss 变小**，但这里有一个致命问题：**❗ Loss 函数只关心“训练数据”，不关心“未来数据”**

所以模型会：**既学规律，也学噪音**

👉 这就是过拟合的根源：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

## 二、用一个贯穿案例讲透（核心）

假设我们有一组数据：

👉 本质规律是： $y \approx x$ ，但每个点都有一点随机误差（噪声）。

### 第一阶段：模型还不会（欠拟合）

模型很简单： $y = \text{常数}$

结果：

- 完全拟合不了
- 误差很大

👉 特点：模型太弱，连规律都没学会

### 第二阶段：模型学到规律（最佳状态）

模型变强一点： $y = ax + b$

结果：

- 能抓住整体趋势
- 不受个别点影响

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

👉 这是最理想状态：学规律，不学噪声

### 第三阶段：模型继续学习（过拟合开始）

模型继续优化，会发生什么？

它会发现：“这些点不是完全在直线上，我可以再优化一下”

于是模型开始：

- 调整曲线去贴每一个点
- 包括那些“随机误差”

👉 最终变成：一条非常扭曲的曲线，穿过所有点

👉 这时候：

- 训练集：几乎完美 ✓
- 测试集：表现变差 !

👉 本质：

模型把“噪声”当成了“规律”

## 三、为什么训练越久，越容易过拟合？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

这个问题非常关键。

## 原因1：模型容量 (Model Capacity)

模型越复杂（参数越多）：

- 能表达的函数越复杂
- 越容易“记住数据”

👉 类比：

- 小学生 → 学规律
- 记忆大师 → 可以背整本书

👉 深度网络就是“记忆能力极强的模型”

## 原因2：训练时间 (Epoch 太多)

训练时间越长：

- 模型优化得越极致
- 越容易把噪声也学进去

👉 对应图：Epoch 增加 → 验证 Loss 开始上升

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

### 原因3：数据量不够

数据太少时：

- 噪声占比变大
- 模型更容易“误判规律”

👉 一句话：

**数据少 + 模型强 → 最容易过拟合**

### 原因4：数据有噪声

现实数据不是完美的：

- 标注错误
- 随机波动

模型不会区分：

- 哪些是规律
- 哪些是噪声

## 四、再从“泛化能力📺”角度理解（关键提升）

什么是泛化能力？模型在“新数据”上的表现能力

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

过拟合的本质就是：

- 在训练集上很好
- 在新数据上变差

数学上可以理解为：

$$\mathcal{L}_{train} \downarrow, \quad \mathcal{L}_{test} \uparrow$$

核心矛盾：训练能力 (fit) vs 泛化能力 (generalization)

## 五、用代码视角再理解一次（工程感）

一个典型训练过程：

```
go AI写代码 | 运行  
1 | for epoch in range(epochs):  
2 |     train_loss = model.train(train_data)  
3 |     val_loss = model.evaluate(val_data)
```

你会观察到：

- Epoch 1: train ↓ val ↓
- Epoch 5: train ↓ val ↓
- Epoch 10: train ↓ val ↑ !

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

如果你只看 `train_loss`: **你会以为模型越来越好**

但 `val_loss` 在告诉你: **模型已经开始学歪了**

## 六、回到这张图，再看一遍（彻底理解）

现在你再看图：

### 上半部分（Loss 曲线）

- 蓝线：模型在“记住数据”
- 红线：泛化能力下降

### 下半部分（拟合图）

- 欠拟合：不会
- 正常拟合：刚刚好
- 过拟合：学太多

👉 关键不是“拟合多好”，而是：**是否还能适用于新数据**

你可以只记住这三句话：

- 1 模型训练的目标是降低训练误差，但不保证泛化能力
- 2 模型越强、训练越久，越容易记住噪声
- 3 过拟合 = 学会了数据细节，但丢失了泛化能力

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: [https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页: [https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

再给你一句最核心的：**模型不是学得越多越好，而是学对才重要**

## 七、什么是噪音？

在**机器学习** 中，**噪音 (Noise)** 指的是：

- 数据中那些随机的、不稳定的、不能代表真实规律的信息。

模型真正**应该学的是“规律”，而不是“噪音”**。但问题在于，模型并不知道谁是规律，谁是噪音，它只会尽量把训练误差降到最低。于是当**模型过强、训练过久时，就可能把噪音也一起学进去，这就会导致过拟合**。

**常见的噪音来源有几种：**

第一种是**测量误差**。

比如温度**传感器** 本来应该读到 25°C，但因为设备误差显示成了 25.6°C。

第二种是**标注错误**。

比如一张明明是“猫”的图片，被人工误标成了“狗”。

第三种是**随机波动**。

比如房价整体和面积相关，但某一套房因为装修、楼层、采光等偶然因素，比正常价格高很多。

第四种是**无关信息干扰**。

比如识别猫狗图片时，模型不小心把“背景草地”也当成了判断依据，但草地本身并不是猫的本质特征。

## 八、为什么噪音会导致过拟合？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

因为模型训练时只会想办法让训练集 Loss 变小。如果模型容量足够大，它不仅能学会“主要规律”，还可能把那些偶然偏差也硬记下来。

举个简单例子，假设真实规律大致是： $y=x$ ，但训练数据里某些点因为误差，偏离了这条趋势线。

- 如果模型太简单，它可能抓不住规律；
- 如果模型刚刚好，它会学到“整体趋势”；
- 如果模型太复杂，它就会努力把每个偏离点都拟合进去。

这时候模型学到的就不再只是规律，而是：**规律 + 噪音**

于是它在训练集上表现很好，但到了新数据上就不行了，因为新数据里的噪音不会和训练集一模一样。

一句话总结

**噪音就是数据里那些不能重复出现、也不该被模型当成规律学习的随机信息。**

## 八、如何防止过拟合（正则化 / Dropout / Early Stopping）

过拟合的本质是：

**模型记住了训练数据的细节和噪音，却没有真正学会可泛化的规律。**

所以，防止过拟合的方法，本质上都在做一件事：**限制模型“死记硬背”的能力，逼它去学更稳定的规律。**

下面这三种方法最常见。

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

## 1. 正则化 (Regularization)

正则化的思路可以理解成：给模型加一个“约束”，不要让参数变得太夸张。

在训练时，我们原本只优化数据误差：

$$Loss = Loss_{data}$$

加上正则化之后，就变成：

$$Loss = Loss_{data} + \lambda \cdot Regularization$$

如果是最常见的 L2 正则化，可以写成：

$$Loss = Loss_{data} + \lambda \sum w^2$$

这里的意思是：

- 第一部分：让模型尽量拟合数据
- 第二部分：惩罚过大的权重参数

——为什么这能减少过拟合？——

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

- 因为参数越大，模型往往更容易做出非常复杂、弯弯绕绕的拟合曲线，也更容易把噪音记进去。
- 加了正则化之后，模型会更倾向于用“更平滑、更简单”的方式去拟合数据，而不是为了追求训练集完美贴合，把自己搞得过于复杂。

你可以把正则化理解成：**给模型套一个“不要太放飞”的约束。**

## 2. Dropout

Dropout 的思路非常直观：**训练时随机让一部分神经元“暂时下线”。**

比如某一层有很多神经元，训练时随机关闭其中一部分，只让剩下的继续工作。下一次训练时，又随机关闭另一部分。

**——这样做的作用是什么？——**

如果不做 Dropout，模型可能会过度依赖某几个特别“好用”的神经元。

一旦这些神经元学到了一些训练集特有的细节，模型就会越来越容易过拟合。

而 Dropout 会迫使模型：

- 不能总依赖固定的少数神经元
- 必须学会更分散、更稳健的特征表达
- 不容易把某些偶然噪音绑定到单一路径上

你可以把 Dropout 理解成：**训练时故意“打乱协作”，防止模型形成死记硬背的小团体。**

内容来源：csdn.net

作者：快乐飞猪ing

原文链接：https://blog.csdn.net/m0\_46279060/article/details/159589443

作者主页：https://blog.csdn.net/m0\_46279060

所以它的本质是：**降低神经元之间的过度依赖，提升模型泛化能力。**

### 3. Early Stopping

Early Stopping 的思路最容易理解：

**不要等模型开始学歪了才停，要在它刚刚最好的时候停下来。**

回到你那张图：

- 训练集 Loss 一直下降
- 验证集 Loss 先下降后上升

当验证集 Loss 开始上升时，其实就是在提醒你：**模型已经开始记住训练数据细节，泛化能力在变差了**

Early Stopping 的做法就是：

- 一边训练
- 一边监控验证集表现
- 一旦发现验证集不再变好，甚至开始变差
- 就停止训练

它的核心思想不是“训练得越久越好”，而是：**在模型泛化能力最好的那个点停下**，所以 Early Stopping 本质上是：**用验证集帮你判断，模型有没有开始过拟合。**

这三种方法分别在解决什么问题？你可以这样理解：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)

**正则化：限制模型不要太复杂**

**Dropout：防止模型过度依赖局部特征**

**Early Stopping：在模型开始学歪之前及时停下**

它们方向不同，但目的相同：**让模型更难记住噪音，更容易学到真正可泛化的规律。**

一个总结：

**防止过拟合的核心，不是让模型“不学习”，而是让模型“别学得太细、太死、太偏”。**

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：[https://blog.csdn.net/m0\\_46279060/article/details/159589443](https://blog.csdn.net/m0_46279060/article/details/159589443)

作者主页：[https://blog.csdn.net/m0\\_46279060](https://blog.csdn.net/m0_46279060)